LC 485 — Max Consecutive Ones

Given a binary array, return the maximum number of consecutive 1s.

Input: [1,1,0,1,1,1]
Output: 3

```
function maxConsecutiveOnes(nums) {
  let count = 0, best = 0;
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === 1) {
      count++;
      if (count > best) best = count;
    } else {
      count = 0;
    }
  }
  return best;
}
```

LC 3 — Longest Substring Without Repeating Characters

Return the length of the longest substring without repeating characters.

Input: "abcabcbb"
Output: 3

```
function lengthOfLongestSubstring(s) {
  const map = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    map.set(ch, (map.get(ch) || 0) + 1);
    while (map.get(ch) > 1) {
      const out = s[left];
      map.set(out, map.get(out) - 1);
      if (map.get(out) === 0) map.delete(out);
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

LC 904 — Fruits Into Baskets

Find the length of the longest subarray containing at most two distinct values.

Input: [1,2,1]
Output: 3

```
function totalFruit(fruits) {
  const map = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < fruits.length; right++) {
    const x = fruits[right];
    map.set(x, (map.get(x) || 0) + 1);
    while (map.size > 2) {
      const y = fruits[left++];
      map.set(y, map.get(y) - 1);
      if (map.get(y) === 0) map.delete(y);
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

LC 340 — Longest Substring with At Most K Distinct (Premium)

Return the length of the longest substring with at most K distinct characters.

Input: "eceba", K=2
Output: 3

```
function lengthOfLongestSubstringKDistinct(s, k) {
  const map = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    map.set(ch, (map.get(ch) || 0) + 1);
    while (map.size > k) {
      const out = s[left++];
      map.set(out, map.get(out) - 1);
      if (map.get(out) === 0) map.delete(out);
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

LC 209 — Minimum Size Subarray Sum

Given target and positive nums, return the minimal length of a subarray with sum >= target.

Input: target=7, nums=[2,3,1,2,4,3]
Output: 2

```
function minSubArrayLen(target, nums) {
  let left = 0, sum = 0, best = Infinity;
  for (let right = 0; right < nums.length; right++) {
    sum += nums[right];
    while (sum >= target) {
      best = Math.min(best, right - left + 1);
      sum -= nums[left++];
      if (best === 1) return 1;
    }
  }
  return best === Infinity ? 0 : best;
}
```

LC 713 — Subarray Product Less Than K

Count the number of contiguous subarrays where the product of all elements is < k (nums positive).

Input: nums=[10,5,2,6], k=100
Output: 8

```
function numSubarrayProductLessThanK(nums, k) {
  if (k <= 1) return 0;
  let left = 0, prod = 1, count = 0;
  for (let right = 0; right < nums.length; right++) {
    prod *= nums[right];
    while (prod >= k) prod /= nums[left++];
    count += right - left + 1;
  }
  return count;
}
```

LC 567 — Permutation in String

Return true if s2 contains any permutation of s1.

Input: s1="ab", s2="eidbaooo"
Output: true

```
function checkInclusion(s1, s2) {
  if (s1.length > s2.length) return false;
  const need = new Array(26).fill(0);
  const A = c => c.charCodeAt(0) - 97;
  for (let i = 0; i < s1.length; i++) need[A(s1[i])]++;
  let left = 0;
  for (let right = 0; right < s2.length; right++) {
    const r = A(s2[right]);
    need[r]--;
    while (need[r] < 0) {
      const l = A(s2[left]);
      need[l]++;
      left++;
    }
    if (right - left + 1 === s1.length) return true;
  }
  return false;
}
```

LC 1004 — Max Consecutive Ones III

Return the length of the longest subarray containing at most K zeros.

Input: nums=[1,1,1,0,0,0,1,1,1,1,0], K=2
Output: 6

```
function longestOnes(nums, K) {
  let left = 0, zeros = 0, best = 0;
  for (let right = 0; right < nums.length; right++) {
    if (nums[right] === 0) zeros++;
    while (zeros > K) {
      if (nums[left] === 0) zeros--;
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

LC 438 — Find All Anagrams in a String

Return all start indices of p's anagrams in s.

Input: s="cbaebabacd", p="abc"
Output: [0,6]

```
function findAnagrams(s, p) {
  const m = p.length, n = s.length;
  if (m > n) return [];
  const need = new Array(26).fill(0), win = new Array(26).fill(0);
  const A = c => c.charCodeAt(0) - 97;
  for (let i = 0; i < m; i++) need[A(p[i])]++;
  const res = [];
  for (let i = 0; i < n; i++) {
    win[A(s[i])]++;
    if (i >= m) win[A(s[i - m])]--;
    let ok = true;
    for (let j = 0; j < 26; j++) {
      if (win[j] !== need[j]) { ok = false; break; }
    }
    if (ok) res.push(i - m + 1);
  }
  return res;
}
```

LC 76 — Minimum Window Substring

Return the smallest window in s that contains all chars of t.

Input: s="ADOBECODEBANC", t="ABC"
Output: "BANC"

```
function minWindow(s, t) {
  const need = new Map();
  for (let i = 0; i < t.length; i++) {
    const c = t[i];
    need.set(c, (need.get(c) || 0) + 1);
  }
  let missing = t.length, left = 0, bestL = 0, bestR = Infinity;
  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    if (need.has(c)) {
      if (need.get(c) > 0) missing--;
      need.set(c, need.get(c) - 1);
    }
    while (missing === 0) {
      if (right - left < bestR - bestL) { bestL = left; bestR = right; }
      const d = s[left++];
      if (need.has(d)) {
        need.set(d, need.get(d) + 1);
        if (need.get(d) > 0) missing++;
      }
    }
  }
  return bestR === Infinity ? "" : s.slice(bestL, bestR + 1);
}
```

LC 424 — Longest Repeating Character Replacement

Return the length of the longest substring you can get by replacing at most K characters to make all chars equal.

Input: s="AABABBA", K=1
Output: 4

```
function characterReplacement(s, K) {
  const freq = new Array(26).fill(0);
  let left = 0, best = 0, maxFreq = 0;
  for (let right = 0; right < s.length; right++) {
    const i = s.charCodeAt(right) - 65;
    freq[i]++;
    if (freq[i] > maxFreq) maxFreq = freq[i];
    while ((right - left + 1) - maxFreq > K) {
      const j = s.charCodeAt(left) - 65;
      freq[j]--;
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Longest Subarray with Sum ≤ S (non-negatives)

Return the length of the longest subarray with sum ≤ S for non-negative nums.

Input: nums=[1,2,1,1,1], S=3
Output: 3

```
function longestSubarraySumAtMost(nums, S) {
  let left = 0, sum = 0, best = 0;
  for (let right = 0; right < nums.length; right++) {
    sum += nums[right];
    while (sum > S) sum -= nums[left++];
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

LC 930 — Binary Subarrays With Sum

Count subarrays with sum equal to S in a binary array using the atMost trick.

Input: nums=[1,0,1,0,1], S=2
Output: 4

```
function numSubarraysWithSum(nums, S) {
  function atMost(T) {
    if (T < 0) return 0;
    let left = 0, sum = 0, ans = 0;
    for (let right = 0; right < nums.length; right++) {
      sum += nums[right];
      while (sum > T) sum -= nums[left++];
      ans += right - left + 1;
    }
    return ans;
  }
  return atMost(S) - atMost(S - 1);
}
```

LC 1493 — Longest Subarray of 1s After Deleting One Element

Return the length of the longest subarray of 1s after deleting exactly one element.

Input: [1,1,0,1]
Output: 3

```
function longestSubarray(nums) {
  let left = 0, zeros = 0, best = 0;
  for (let right = 0; right < nums.length; right++) {
    if (nums[right] === 0) zeros++;
    while (zeros > 1) {
      if (nums[left] === 0) zeros--;
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best - 1;
}
```